



برنامه نویسی شی گرا

Object Oriented Programming

مدرس:

مریم خراشادیزاده

وراثت

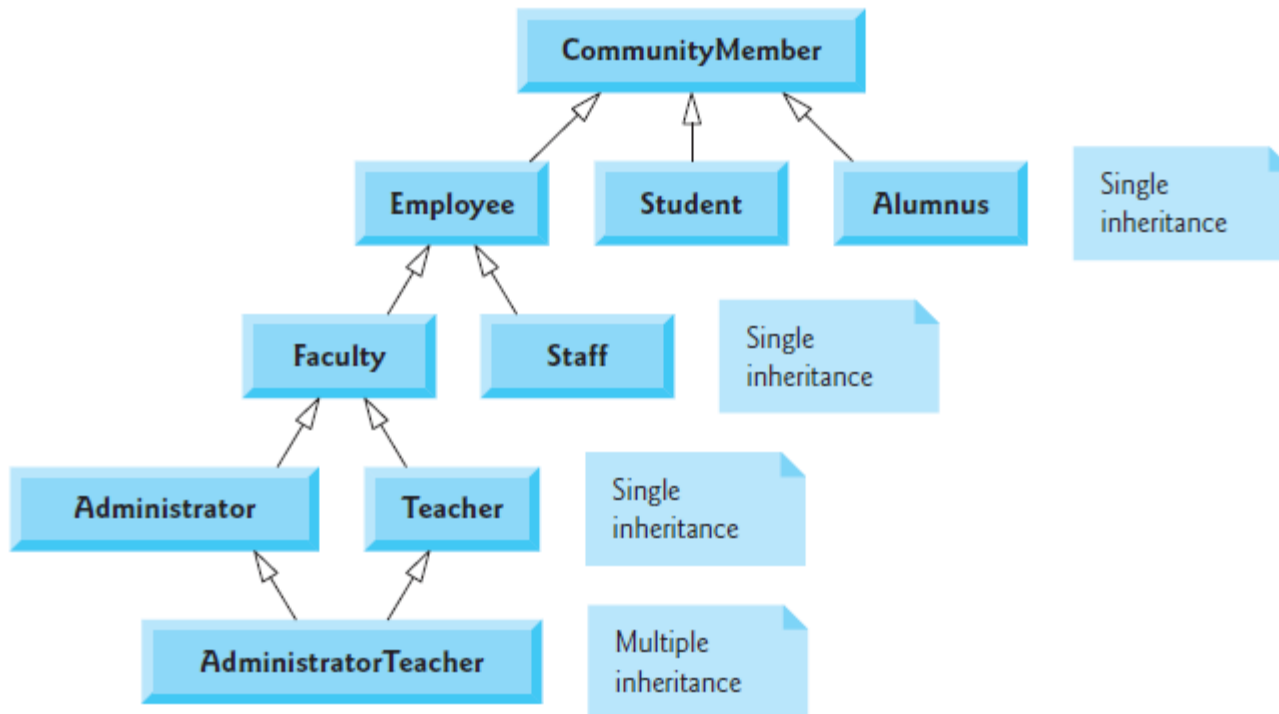
Inheritance

- همان‌طور که در ابتدای بحث شی‌گرایی بیان شد، سبک برنامه‌نویسی شی‌گرا از دنیای واقعی الهام گرفته شده است. در این سبک برنامه‌نویسی یکی از اصول پایه مفهوم **توارث** است که باز هم از دنیای واقعی وام گرفته شده است. در دنیای واقعی فرزندان ویژگیهایی را از والدین خود به ارث می‌برند ولی می‌توانند ویژگی‌های جدیدی هم داشته باشند که از والدین آنها گرفته نشده باشد.
- **رابطه ارث‌بری** در سبک برنامه‌نویسی شی‌گرا به این معنی است که یک کلاس توانمندیهای یک کلاسی که قبلاً ایجاد شده است، را در بگیرد و سپس این توانمندیها را افزایش یا بهبود بدهد و یا حتی آنها را برای کاربردی خاصتر جایگزین کند. اساساً یکی از قدرتهای وراثت همین قابلیت افزودن، جایگزینی و پالایش قابلیت‌های به ارث رسیده است.
- در واقع توارث یکی از صورتهای **استفاده مجدد** است. مسلماً استفاده مجدد از نرم‌افزارهای با کیفیت که قبلاً ایجاد، اشکالزدایی شده‌اند و صحت عملکرد آنها تایید شده است، بسیار مطلوب است.
- معمولاً به کلاسی که می‌خواهیم قابلیت‌های آن را به ارث ببریم **کلاس پایه** (base class) و یا **ابرکلاس** (super class) و به کلاسی جدیدی که قابلیت‌های کلاس پایه را به ارث می‌برد **کلاس مشتق شده** (derived class) و یا **زیرکلاس** (sub class) نامیده می‌شود. به عنوان نمونه می‌توان وسیله نقلیه را یک کلاس پایه یا ابرکلاس به حساب آورد و ماشین، اتوبوس، کامیون، قایق، هواپیما، قطار و حتی دوچرخه را کلاسهای مشتق شده یا زیرکلاس این کلاس پایه دانست. معمولاً کلاس پایه عمومی‌تر است یعنی دسته بزرگتری از اشیا را در برمی‌گیرد و کلاس مشتق شده خاصتر است و دسته کوچکتری را از اشیا در بر می‌گیرد.
- رابطه ارث‌بری را یک رابطه **is-a** نیز نامیده می‌شود (در مقابل رابطه ترکیب که یک رابطه **has-a** نامیده می‌شود). زیر یک شی از کلاس مشتق شده شیئی از نوع کلاس پایه است و می‌تواند به عنوان شیئی از کلاس پایه رفتار کند. به عنوان مثال می‌توان گفت که ماشین یک وسیله نقلیه است (Car **is a** Vehicle.) به عبارت دیگر شیء ماشین تمام ویژگیها و رفتارهای شیء وسیله نقلیه را دارد. در صورتی که یک ماشین پدال ترمز دارد (Car **has a** brake pedal.).



سلسه مراتب کلاسه

- یک کلاس می‌تواند یک زیر کلاس باشد، در حالی که کلاسه‌های دیگری از این کلاس ارث‌بری می‌کنند، به عبارت دیگر کلاسه‌های موجود در یک سامانه می‌توانند یک سلسله مراتب (class hierarchy) از ارتباطات ارث‌بری داشته باشند.
- یک کلاس می‌تواند از یک کلاس پایه ارث ببرد که به آن ارث‌بری یگانه (Single inheritance) می‌گویند و یا از چندین کلاس پایه ارث‌بری کند که به آن ارث‌بری چندگانه (Multiple inheritance) می‌گویند.
- یک کلاس پایه می‌تواند کلاس پایه مستقیم یا غیرمستقیم باشد.
- شکل روبرو سلسله مراتب کلاسه‌های در یک دانشگاه نشان می‌دهد.



مثال ۱: وراثت (فایل Base.h)

```
1  #ifndef BASE_H
2  #define BASE_H
3
4  class Base
5  {
6      public:
7          Base(int = 0, int = 0);
8          void Seti(int a) {i = a;}
9          int Geti() const {return i;}
10         void Setj(int a) {j = a;}
11         int Getj() const {return j;}
12         void show() const;
13         ~Base();
14     protected: ← سطح دسترسی حفاظت شده
15         int j;
16     private:
17         int i;
18 };
19 #endif // BASE_H
```



مثال ۱: وراثت (فایل Base.cpp)

```
1  #include "Base.h"
2  #include <iostream>
3
4  using namespace std;
5
6  Base::Base(int a, int b):i(a),j(b)
7  {
8      cout << "A base object is constructed: i = " << i;
9      cout << ", j = " << j << endl;
10 }
11 void Base::show() const
12 {
13     cout << i << '-' << j << endl;
14 }
15
16 Base::~Base()
17 {
18     cout << "A base object is destructed: i = " << i;
19     cout << ", j = " << j << endl;
20 }
```



مثال ۱: وراثت (فایل Derived.h)

```
1  #ifndef DERIVED_H
2  #define DERIVED_H
3
4  #include "Base.h"
5
6  class Derived:public Base ← ارث‌بری عمومی از کلاس Base
7  {
8      public:
9          Derived(int = 1,int = 1,int = 1,int = 1);
10         void Setx(int a){x = a;}
11         int Getx() const {return x;}
12         void Sety(int a){y = a;}
13         int Gety()const {return y;}
14         void show() const;
15         ~Derived();
16
17     protected: ← سطح دسترسی حفاظت شده
18         int y;
19
20     private:
21         int x;
22 };
23 #endif // DERIVED_H
```



مثال ۱: وراثت (فایل Derived.cpp)

```
1  #include "Derived.h"
2  #include <iostream>
3
4  using namespace std;
5
6  Derived::Derived(int a,int b,int c,int d):Base(a,b),x(c),y(d)
7  {
8      cout << "A derived object is constructed: x = " << x;
9      cout << ", y = " << y << endl;
10 }
11
12 void Derived::show() const
13 {
14     cout << Geti() << '-' << j << '-';
15     //Base::show();
16     cout << x << '-' << y << endl;
17 }
18 Derived::~Derived()
19 {
20     cout << "A derived object is destructed: x = " << x;
21     cout << ", y = " << y << endl;
22 }
```

- **نکته ۱:** در سازنده کلاس پایه می‌توان با آوردن نام کلاس پایه بعد از : و ارسال آرگومانهای مورد نیاز سازنده کلاس پایه را صریحا صدا زد.
- **نکته ۲:** دسترسی مستقیم به متغیرهای خصوصی کلاس پایه (متغیر i) در کلاس مشتق شده امکانپذیر نیست، به همین علت از تابع عمومی **Geti()** در سطر ۱۴ استفاده شده است. اما دسترسی مستقیم به متغیرهای حفاظت شده کلاس پایه (متغیر j) در کلاس مشتق شده امکانپذیر است. (سطر ۱۵)
- **نکته ۳:** می‌توان به جای سطر ۱۴ از تابع **show** کلاس پایه (سطر ۱۵) را فراخوانی کرد. توابع کلاس پایه که در کلاس مشتق شده بازنویسی شده‌اند (مثل تابع **show**) را با استفاده از نام کلاس پایه و عملگر :: می‌توان فراخوانی کرد.



مثال ۱: وراثت (فایل main.cpp)

```
1 #include <iostream>
2 #include "Derived.h"
3
4 using namespace std;
5
6 int main()
7 {
8     Base b1(1,2), *Baseptr,b2;
9     Derived d1(10,20,30,40), *Derivedptr,d2;
10
11     cout << " b1: "; b1.show();
12     cout<<"-----\n";
13     cout << " b2: "; b2.show();
14     cout<<"-----\n";
15     cout << " d1: "; d1.show();
16     cout<<"-----\n";
17     cout << " d2: "; d2.show();
18     cout<<"-----\n";
19     Baseptr = &b1;
20     cout << " Baseptr: "; Baseptr -> show();
21     cout<<"-----\n";
22     b2 = d1;
23     cout << " b2: "; b2.show();
24     //d2 = b1;      Error
25     cout<<"-----\n";
26     d2.Seti(11);
27     d2.Setj(12);
28     d2.Setx(13);
29     d2.Sety(14);
30     Derivedptr = &d2;
31     cout << " Derivedptr: "; Derivedptr -> show();
32     cout<<"-----\n";
33     Baseptr = &d2;
34     cout << " Baseptr: "; Baseptr->show();
35     cout<<"-----\n";
36     Derivedptr = static_cast<Derived *>(&b2);
37     cout << " Derivedptr: "; Derivedptr ->show();
38     cout<<"-----\n";
39     Baseptr=&d1;
40     Derivedptr = static_cast<Derived *>(Baseptr);
41     cout << " Derivedptr: "; Derivedptr ->show();
42     cout<<"-----\n";
43
44     return 0;
45 }
```

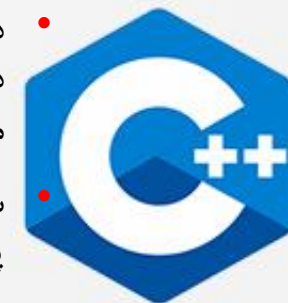
```
"C:\Users\Maryam\courses\Advanced programing\Inheritan...
A base object is constructed: i = 1, j = 2
A base object is constructed: i = 0, j = 0
A base object is constructed: i = 10, j = 20
A derived object is constructed: x = 30, y = 40
A base object is constructed: i = 1, j = 1
A derived object is constructed: x = 1, y = 1
b1: 1-2
-----
b2: 0-0
-----
d1: 10-20-30-40
-----
d2: 1-1-1-1
-----
Baseptr: 1-2
-----
b2: 10-20
-----
Derivedptr: 11-12-13-14
-----
Baseptr: 11-12
-----
Derivedptr: 10-20-1-2
-----
Derivedptr: 10-20-30-40
-----
A derived object is destructed: x = 13, y = 14
A base object is destructed: i = 11, j = 12
A derived object is destructed: x = 30, y = 40
A base object is destructed: i = 10, j = 20
A base object is destructed: i = 10, j = 20
A base object is destructed: i = 1, j = 2
Process returned 0 (0x0)   execution time : 0.187 s
Press any key to continue.
```

وراثت و سطوح دسترسی

- در C++ وراثت به سه شکل **عمومی** (public)، **حفاظت شده** (protected) و **خصوصی** (private) وجود دارد که معمولترین آن حالت عمومی است که به شکل زیر اعلان می‌شود. در صورت اعلان ارث‌بری به حالت حفاظت شده و خصوصی از کلمات کلیدی protected و private به جای public استفاده می‌شود.

`class DerivedClassName: public BaseClassName`

- در هر حالت ارث‌بری (عمومی، حفاظت شده و یا خصوصی) **اعضای عمومی کلاس پایه** به صورت مستقیم قابل دسترسی در کلاس مشتق شده **هستند**.
- در هر حالت ارث‌بری (عمومی، حفاظت شده و یا خصوصی) **اعضای خصوصی کلاس پایه** به صورت مستقیم قابل دسترسی در کلاس مشتق شده **نیستند**. دقت کنید که در هر صورت اعضای خصوصی کلاس پایه نیز به کلاسهای مشتق شده به ارث می‌رسند، اما به این اعضا به صورت غیرمستقیم و با استفاده از اعضای غیرخصوصی (مثلا توابع عمومی) می‌توان دسترسی پیدا کرد.
- **سطح دسترسی حفاظت شده (protected)** سطح دسترسی جدیدی است که در رابطه با ارث‌بری مطرح می‌شود و حالتی بینابین سطح دسترسی عمومی و خصوصی است. اعضای که به صورت حفاظت شده در کلاسی معرفی می‌شوند توسط اعضای خود کلاس و دوستان کلاس و همچنین کلاسهای مشتق شده و دوستان کلاسهای مشتق شده قابل دسترسی هستند.
- در هر حالت ارث‌بری (عمومی، حفاظت شده و یا خصوصی) **اعضای حفاظت شده کلاس پایه** به صورت مستقیم قابل دسترسی در کلاس مشتق شده **هستند**.
- دقت کنید برای ایجاد کدی غیرشکننده با امکان پشتیبانی راحتتر، شاید استفاده از سطح دسترسی خصوصی برای اعضای داده‌ای صحیح‌تر باشد. زیرا سطح دسترسی حفاظت شده امکان دستکاری داده‌های حفاظت شده کلاس پایه را به صورت مستقیم در کلاسهای مشتق شده فراهم می‌کند که این کار امکان مقاداردهیهای غیرمجاز را در کلاسهای مشتق شده فراهم می‌کند و ضمناً تغییرات در کلاس پایه ممکن است مستلزم تغییرات در کلاسهای مشتق شده شود.
- شیء کلاس مشتق شده نوعی از کلاس پایه است پس می‌توان شیء کلاس مشتق شده به متغیری از نوع کلاس پایه نسبت داد و یا با اشاره‌گری از نوع کلاس پایه به آن اشاره کرد؛ ولی عکس آن درست نیست.



وراثت و سطوح دسترسی

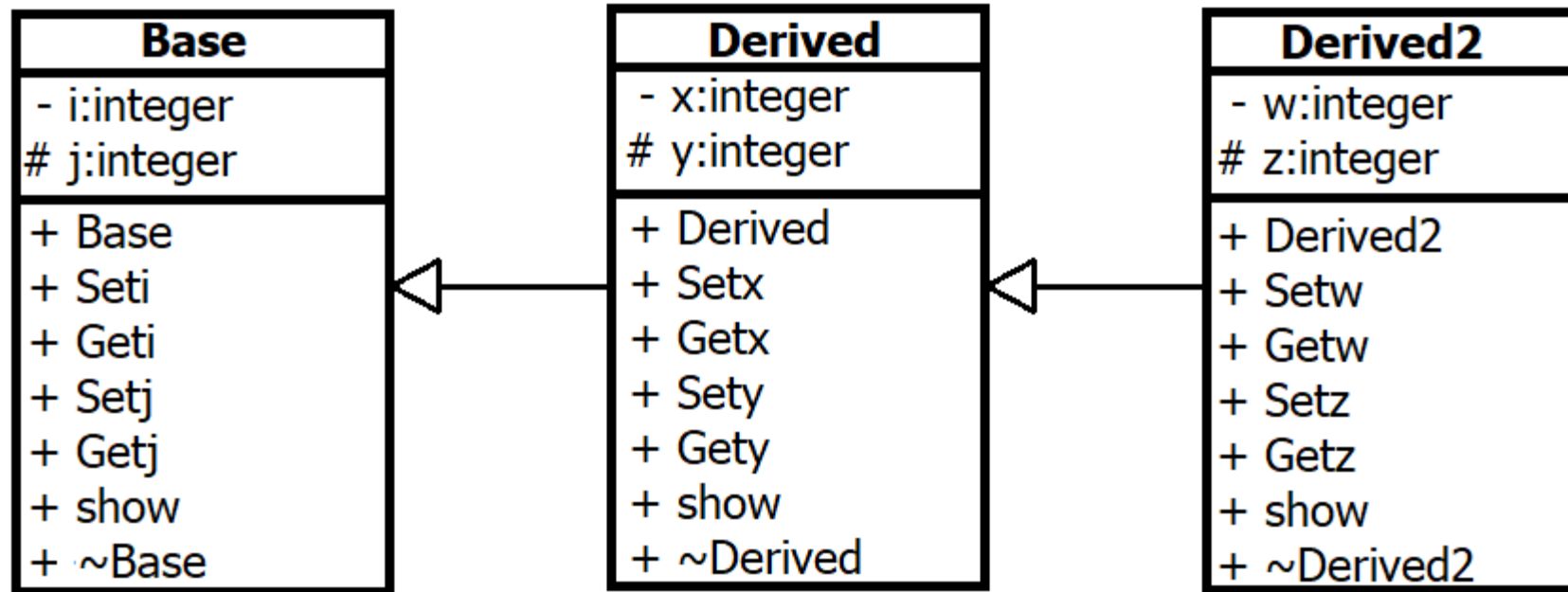
• جدول زیر به طور خلاصه ارتباط بین سطح دسترسی اعضای کلاس پایه و همچنین نوع ارث‌بری را نشان می‌دهد:

Base-class member-access specifier	Type of inheritance		
	public inheritance	protected inheritance	private inheritance
public	public in derived class. Can be accessed directly by member functions, friend functions and nonmember functions.	protected in derived class. Can be accessed directly by member functions and friend functions.	private in derived class. Can be accessed directly by member functions and friend functions.
protected	protected in derived class. Can be accessed directly by member functions and friend functions.	protected in derived class. Can be accessed directly by member functions and friend functions.	private in derived class. Can be accessed directly by member functions and friend functions.
private	Hidden in derived class. Can be accessed by member functions and friend functions through public or protected member functions of the base class.	Hidden in derived class. Can be accessed by member functions and friend functions through public or protected member functions of the base class.	Hidden in derived class. Can be accessed by member functions and friend functions through public or protected member functions of the base class.



وراثت و سطوح دسترسی

• مثال ۲: می‌خواهیم سلسله مراتب ارث‌بری زیر را پیاده‌سازی کنیم.



مثال ۲: وراثت (فایل Base.h)

```
1  #ifndef BASE_H
2  #define BASE_H
3
4  class Base
5  {
6      public:
7          Base(int = 0, int = 0);
8          void Seti(int a) {i = a;}
9          int Geti() const {return i;}
10         void Setj(int a) {j = a;}
11         int Getj() const {return j;}
12         void show() const;
13         ~Base();
14     protected:
15         int j;
16     private:
17         int i;
18 };
19 #endif // BASE_H
```



مثال ۱: وراثت (فایل Base.cpp)

```
1  #include "Base.h"
2  #include <iostream>
3
4  using namespace std;
5
6  Base::Base(int a, int b):i(a),j(b)
7  {
8      cout << "A base object is constructed: i = " << i;
9      cout << ", j = " << j << endl;
10 }
11 void Base::show() const
12 {
13     cout << i << '-' << j << endl;
14 }
15
16 Base::~Base()
17 {
18     cout << "A base object is destructed: i = " << i;
19     cout << ", j = " << j << endl;
20 }
```



مثال ۲: وراثت (فایل Derived.h)

```
1  #ifndef DERIVED_H
2  #define DERIVED_H
3
4  #include "Base.h"
5
6  class Derived:public Base ← ارث‌بری عمومی از کلاس Base
7  {
8      public:
9          Derived(int = 1,int = 1,int = 1,int = 1);
10         void Setx(int a){x = a;}
11         int Getx() const {return x;}
12         void Sety(int a){y = a;}
13         int Gety()const {return y;}
14         void show() const;
15         ~Derived();
16
17     protected:
18         int y;
19
20     private:
21         int x;
22 };
23 #endif // DERIVED_H
```



مثال ۱: وراثت (فایل Derived.cpp)

```
1  #include "Derived.h"
2  #include <iostream>
3
4  using namespace std;
5
6  Derived::Derived(int a,int b,int c,int d):Base(a,b),x(c),y(d)
7  {
8      cout << "A derived object is constructed: x = " << x;
9      cout << ", y = " << y << endl;
10 }
11
12 void Derived::show() const
13 {
14     cout << Geti() << '-' << j << '-';
15     //Base::show();
16     cout << x << '-' << y << endl;
17 }
18 Derived::~Derived()
19 {
20     cout << "A derived object is destructed: x = " << x;
21     cout << ", y = " << y << endl;
22 }
```

مثال ۲: وراثت (فایل Derived2.h)

```
1  #ifndef DERIVED2_H
2  #define DERIVED2_H
3
4  #include "Derived.h"
5
6  class Derived2:public Derived ← ارث‌بری عمومی از کلاس Derived
7  {
8      public:
9          Derived2(int = 2,int = 2, int = 2,int = 2,int = 2,int = 2);
10         void Setw(int a){w = a;}
11         int Getw() const {return w;}
12         void Setz(int a){z = a;}
13         int Getz() const {return z;}
14         void show() const;
15         ~Derived2();
16
17     protected:
18         int z;
19
20     private:
21         int w;
22 };
23
24 #endif // DERIVED2_H
```



مثال ۲: وراثت (فایل Derived2.cpp)

```
1  #include "Derived2.h"
2  #include <iostream>
3
4  using namespace std;
5
6  Derived2::Derived2(int a, int b, int c, int d, int e, int f):
7      Derived(a, b, c, d), w(e), z(f)
8  {
9      cout << "A derived2 object is constructed: w = " << w;
10     cout << ", z = " << z << endl;
11 }
12 void Derived2::show() const
13 {
14     cout << Geti() << '-' << j << '-';
15     cout << Getx() << '-' << y << '-';
16     //Derived::show();
17     cout << w << '-' << z << endl;
18 }
19 Derived2::~~Derived2()
20 {
21     cout << "A derived2 object is destructed: w = " << w;
22     cout << ", z = " << z << endl;
23 }
```



مثال ۲: وراثت (فایل main.cpp)

```
1  #include <iostream>
2  #include "Derived2.h"
3  using namespace std;
4
5  int main()
6  {
7      Base b1(100,200), *Baseptr;
8      Derived d1(10,20,30,40), *Derivedptr;
9      Derived2 der2(1,2,3,4,5,6), *Derived2ptr;
10     cout<<"-----\n";
11     cout << " der2: "; der2.show();
12     cout<<"-----\n";
13     Derived2ptr = &der2;
14     cout << " Derived2ptr: "; Derived2ptr -> show();
15     cout<<"-----\n";
16     Derivedptr = &der2;
17     Derivedptr -> Seti(100);
18     Derivedptr -> Setx(200);
19     //Derivedptr -> Setw(300);    Error
20     cout << " Derivedptr: "; Derivedptr -> show();
21     cout<<"-----\n";
22     Baseptr = &der2;
23     Baseptr -> Seti(400);
24     //Baseptr -> Setx(500);    Error
25     //Baseptr -> Setw(600);    Error
26     cout << " Baseptr: "; Baseptr -> show();
27     cout<<"-----\n";
28     b1 = der2;
29     cout << " b1: "; b1.show();
30     //der2 = b1;    Error
31     cout<<"-----\n";
32     d1 = der2;
33     cout << " d1: "; d1.show();
34     //der2 = d1;    Error
35     cout<<"-----\n";
36     Derived2ptr = &der2;
37     Derived2ptr -> Seti(11);
38     Derived2ptr -> Setj(12);
39     Derived2ptr -> Setx(13);
40     Derived2ptr -> Sety(14);
41     Derived2ptr -> Setw(15);
42     Derived2ptr -> Setz(16);
43     cout << " Derived2ptr: "; Derived2ptr -> show();
44     cout<<"-----\n";
45     Derived2ptr = static_cast<Derived2*>(&b1);
46     cout << " Derived2ptr: "; Derived2ptr -> show();
47     cout<<"-----\n";
48     Derived2ptr = static_cast<Derived2*>(&d1);
49     cout << " Derived2ptr: "; Derived2ptr -> show();
50     cout<<"-----\n";
51     return 0;
52 }
```

```
"C:\Users\Maryam\courses\Advanced programing\Inheritan...
A base object is constructed: i = 100, j = 200
A base object is constructed: i = 10, j = 20
A derived object is constructed: x = 30, y = 40
A base object is constructed: i = 1, j = 2
A derived object is constructed: x = 3, y = 4
A derived2 object is constructed: w = 5, z = 6
-----
der2: 1-2-3-4-5-6
-----
Derived2ptr: 1-2-3-4-5-6
-----
Derivedptr: 100-2-200-4
-----
Baseptr: 400-2
-----
b1: 400-2
-----
d1: 400-2-200-4
-----
Derived2ptr: 11-12-13-14-15-16
-----
Derived2ptr: 400-2-0-6487488-0-6487488
-----
Derived2ptr: 400-2-200-4-400-2
-----
A derived2 object is destructed: w = 15, z = 16
A derived object is destructed: x = 13, y = 14
A base object is destructed: i = 11, j = 12
A derived object is destructed: x = 200, y = 4
A base object is destructed: i = 400, j = 2
A base object is destructed: i = 400, j = 2

Process returned 0 (0x0)    execution time : 0.192 s
Press any key to continue.
```



وراثت و سطوح دسترسی

- با توجه به جدول اسلاید ۱۰ که ارتباط بین سطح دسترسی اعضای کلاس پایه و همچنین نوع ارث‌بری را نشان می‌دهد:
- اگر نوع ارث‌بری کلاس Derived از کلاس Base از نوع **public** به **protected** یا **private** تغییر کند، آیا نیاز به تغییر در سایر پیاده‌سازیه‌ها وجود دارد؟

`class Derived: protected Base`

`class Derived: private Base`

- اگر نوع ارث‌بری کلاس Derived2 از کلاس Derived از نوع **public** به **protected** یا **private** تغییر کند، آیا نیاز به تغییر در سایر پیاده‌سازیه‌ها وجود دارد؟

`class Derived2: protected Derived`

`class Derived2: private Derived`

- غیر از خطاهایی که به سبب تغییر سطوح دسترسی با تغییر نوع ارث‌بری رخ می‌دهد، یکی از اساسی‌ترین خطاهایی که رخ می‌دهد این است که **تنها در ارث‌بری عمومی شیء کلاس مشتق شده نوعی از کلاس پایه است**. به عبارت دیگر در صورت ارث‌بری حفاظت شده و یا خصوصی دیگر نمی‌توان شیء کلاس مشتق شده را به کلاس پایه (یا حتی به متغیرهای اشاره‌گر و ارجاع کلاس پایه) نسبت داد.



چندریختی Polymorphism

- در مبحث وراثت توضیح دادیم که در **سلسله مراتبی از ارث‌بری** تمامی اشیا ایجاد شده از کلاسهای این سلسله مراتب را می‌توان نمونه‌ای از کلاس پایه این سلسله‌مراتب تلقی کرد و همانطوری که دیدیم می‌توان برای دستکاری اشیا مشتق شده از متغیرهای اشاره‌گری (یا ارجاع) کلاس پایه استفاده می‌کنیم.
- **چندریختی** در واقع به مفهوم عمومیت بخشی است. هدف از **چندریختی** طراحی و پیاده‌سازی سامانه‌هایی است که به راحتی قابل توسعه باشند. تا زمانی که کلاس‌های جدید بخشی از سلسله مراتب ارث‌بری هستند که برنامه آنها را پردازش می‌کند، می‌توان کلاسهای جدید را با اندک یا بدون تغییر به بخشهای قبلی برنامه اضافه کرد. تنها بخش‌هایی از یک برنامه که باید برای گنجاندن کلاس‌های جدید تغییر یابد، آن‌هایی هستند که مستقیماً به کلاس جدید مرتبط هستند.
- همان‌طور که در مثالهای قبلی دیدیم **در صورت ارث‌بری عمومی** به اشاره‌گری (ارجاعی) از نوع کلاس پایه می‌توان آدرس شی از کلاس مشتق شده را نسبت داد و به کمک این اشاره‌گری می‌توان توابع کلاس پایه را فراخوانی کرد. دقت کنید در فراخوانی توابع همنام **توابع کلاس پایه** یعنی کلاس اشاره‌گر (نه کلاسی که شی از نوع آن است) فراخوانی خواهند شد.
- همانگونه که باز در مثالهای قبلی دیدیم، عکس عمل فوق، یعنی نسبت دادن شی کلاس پایه به اشاره‌گر (ارجاع) از نوع کلاس مشتق شده امکانپذیر نیست و **خطای کامپایلر** را به دنبال خواهد داشت. زیرا اشاره‌گر کلاس مشتق شده می‌تواند توابع کلاس مشتق شده را فراخوانی کند که ممکن است این کار مشکلاتی را به وجود آورد.
- در چندریختی با ایجاد **توابع مجازی** در کلاس پایه و بازنویسی آنها در کلاسهای مشتق شده این امکان فراهم می‌شود که با اشاره‌گرهای (یا ارجاعها) کلاس پایه، به صورت پویا توابع کلاسهای مشتق شده که به آن نسبت داده شده است، فراخوانی شود.



توابع مجازی

Virtual Functions

- فرض کنید می‌خواهیم تابع **show** در مثال قبلی، به صورت **تابع مجازی** تعریف شود. برای این کار کافی است اعلان تابع **show** در کلاس **Base** به صورت زیر تغییر یابد:

virtual void show() const;

- با همین تغییر تابع **show** در کلاس **Base**، این تابع **در تمامی کلاسهای مشتق شده** در سلسله مراتب ارث‌بری **مجازی** تلقی می‌شود؛ حتی اگر در ادامه سلسله مراتب این موضوع صریحاً ذکر نشود. (البته بهتر است برای ایجاد کد خواناتر در سلسله مراتب ارث‌بری در هر بازنویسی تابع مجازی کلمه کلیدی **virtual** ذکر شود.)
- تغییری که مجازی شدن تابع ایجاد می‌کند، این است که اگر به اشاره‌گری (ارجاعی) از نوع کلاس پایه، آدرس شئی از کلاس مشتق شده را نسبت داده شود و به کمک این اشاره‌گر توابع مجازی فراخوانی شود، دیگر **تابع کلاس پایه فراخوانی نمی‌شود**؛ بلکه فراخوانی تابع در زمان اجرا براساس شئی کلاس مشتق شده تعیین می‌شود. به عبارت دیگر یک فراخوانی تابع مجازی به کمک اشاره‌گر کلاس پایه ممکن است بر اساس شئی نسبت داده شده اجرای متفاوتی به دنبال داشته باشد. انتخاب کد اجرا شده در زمان اجرا و بر اساس شئی نسبت داده شده به اشاره‌گر تعیین می‌شود.
- اصطلاحاً اینکه کدام پیاده‌سازی به تابع مجازی فراخوانی شده با اشاره‌گر کلاس پایه مرتبط شود را نوعی **اتصال (انقیاد) پویا (Dynamic binding)** یا **اتصال (انقیاد) دیر هنگام (Late binding)** می‌نامند، زیرا این کار در زمان اجرای برنامه اتفاق می‌افتد. این اصطلاح در مقابل اتصال (انقیاد) ایستا (**Static binding**) و یا اتصال (انقیاد) زود هنگام (**Early binding**) مطرح شده است که ارتباط بین فراخوانی و کد پیاده‌سازی شده در زمان کامپایل صورت می‌گیرد. در بیشتر مواقع کامپایلر در هنگام کامپایل مشخص می‌کند که در هر فراخوانی تابع چه کدی باید اجرا شود؛ ولی در مورد توابع مجازی این کار در زمان اجرا انجام می‌شود.



توابع مجازی

Virtual Functions

- اگر در کلاس مشتق شده تابع مجازی کلاس پایه بازنویسی **نشود**، کلاس مشتق شده پیاده‌سازی تابع مجازی کلاس پایه را به ارث خواهد برد.
- برای جلوگیری از خطا، در C++11 می‌توان **کلمه کلیدی override** را در بازنویسی توابع مجازی کلاسهای مشتق شده ذکر کرد و این کار باعث می‌شود که کامپایلر بررسی کند که آیا کلاس پایه، تابع مجازی با همان امضا دارد یا نه و اگر نداشته باشد کامپایلر خطا اعلام خواهد کرد.

virtual void show() const override;

- قبل از C++11، می‌توانستیم هر تابع مجازی از کلاس پایه را در کلاسهای مشتق شده بازنویسی کنیم؛ اما در C++11 می‌توان با قرار دادن **کلمه کلیدی final** در اعلان تابع مجازی در کلاس پایه، جلوی بازنویسی این تابع را در کلاسهای مشتق شده گرفت.

virtual FunctionPrototype final;

- با قرار دادن **کلمه کلیدی final** کنار نام کلاس، حتی می‌توان جلوی ارث‌بری از کلاس را گرفت.

class ClassName final

{

// class body

}



مثال ۳: چندریختی (فایل Base.h)

```
1  #ifndef BASE_H
2  #define BASE_H
3
4  class Base
5  {
6      public:
7          Base(int = 0, int = 0);
8          void Seti(int a) { i = a; }
9          int Geti() const { return i; }
10         void Setj(int a) { j = a; }
11         int Getj() const { return j; }
12         virtual void show() const;
13         ~Base();
14     protected:
15         int j;
16     private:
17         int i;
18 };
19 #endif // BASE_H
20
```

اعلان تابع مجازی show ←

- فایل Base.cpp مشابه اسلاید ۵ است، زیرا در پیاده‌سازی تابع مجازی تغییر به وجود نمی‌آید.



مثال ۳: چندریختی (فایل Derived.h)

```
1  #ifndef DERIVED_H
2  #define DERIVED_H
3
4  #include "Base.h"
5
6  class Derived:public Base
7  {
8      public:
9          Derived(int = 1,int = 1,int = 1,int = 1);
10         void Setx(int a){x = a;}
11         int Getx() const {return x;}
12         void Sety(int a){y = a;}
13         int Gety()const {return y;}
14         virtual void show() const override;
15         // void show() const;
16         ~Derived();
17
18         protected:
19             int y;
20
21         private:
22             int x;
23     };
24 #endif // DERIVED_H
```

← ارث‌بری عمومی از کلاس Base

← اعلان بازنویسی تابع مجازی show

- فایل Derived.cpp مشابه اسلاید ۷ است و همانطور که در اسلاید قبلی گفته شد نیازی به تغییر در پیاده‌سازی وجود ندارد.
- ذکر کلمه کلیدی **virtual** و **override** در کلاسهای مشتق شده برای خوانایی بهتر برنامه و پیشگیری از خطاهای احتمالی است و حذف آن تاثیری در عملکرد برنامه ندارد.



مثال ۳: چندریختی (فایل Derived2.h)

```
1  #ifndef DERIVED2_H
2  #define DERIVED2_H
3
4  #include "Derived.h"
5
6  class Derived2:public Derived ← ارث‌بری عمومی از کلاس Derived
7  {
8      public:
9          Derived2(int = 2,int = 2, int = 2,int = 2,int = 2,int = 2);
10         void Setw(int a){w = a;}
11         int Getw() const {return w;}
12         void Setz(int a){z = a;}
13         int Getz() const {return z;}
14         virtual void show() const override; ← اعلان بازنویسی تابع مجازی show
15         // void show() const;
16         ~Derived2();
17
18     protected:
19         int z;
20
21     private:
22         int w;
23 };
24
25 #endif // DERIVED2_H
```

- فایل Derived2.cpp مشابه اسلاید ۱۵ است و همانطور که در اسلایدهای قبلی گفته شد نیازی به تغییر در پیاده‌سازی وجود ندارد.
- ذکر کلمه کلیدی **virtual** و **override** در کلاسهای مشتق شده برای خوانایی بهتر برنامه و پیشگیری از خطاهای احتمالی است و حذف آن تاثیری در عملکرد برنامه ندارد.



مثال ۳: چندریختی (فایل main.cpp)

```
1  #include <iostream>
2  #include "Derived2.h"
3  using namespace std;
4
5  int main()
6  {
7      Base b1(100,200), *Baseptr;
8      Derived d1(10,20,30,40), *Derivedptr;
9      Derived2 der2(1,2,3,4,5,6), *Derived2ptr;
10     cout<<"-----\n";
11     cout << " der2: "; der2.show();
12     cout<<"-----\n";
13     Derived2ptr = &der2;
14     cout << " Derived2ptr: "; Derived2ptr -> show();
15     cout<<"-----\n";
16     Derivedptr = &der2;
17     Derivedptr -> Seti(100);
18     Derivedptr -> Setx(200);
19     //Derivedptr -> Setw(300);    Error
20     cout << " Derivedptr: "; Derivedptr -> show();
21     cout<<"-----\n";
22     Baseptr = &der2;
23     Baseptr -> Seti(400);
24     //Baseptr -> Setx(500);    Error
25     //Baseptr -> Setw(600);    Error
26     cout << " Baseptr: "; Baseptr -> show();
27     cout<<"-----\n";
28     b1 = der2;
29     cout << " b1: "; b1.show();
30     //der2 = b1;    Error
31     cout<<"-----\n";
32     d1 = der2;
33     cout << " d1: "; d1.show();
34     //der2 = d1;    Error
35     cout<<"-----\n";
36     Derived2ptr = &der2;
37     Derived2ptr -> Seti(11);
38     Derived2ptr -> Setj(12);
39     Derived2ptr -> Setx(13);
40     Derived2ptr -> Sety(14);
41     Derived2ptr -> Setw(15);
42     Derived2ptr -> Setz(16);
43     cout << " Derived2ptr: "; Derived2ptr -> show();
44     cout<<"-----\n";
45     Derived2ptr = static_cast<Derived2*>(&b1);
46     cout << " Derived2ptr: "; Derived2ptr -> show();
47     cout<<"-----\n";
48     Derived2ptr = static_cast<Derived2*>(&d1);
49     cout << " Derived2ptr: "; Derived2ptr -> show();
50     cout<<"-----\n";
51     return 0;
52 }
```

```
"C:\Users\Maryam\courses\Advanced programing\Inheritan...
A base object is constructed: i = 100, j = 200
A base object is constructed: i = 10, j = 20
A derived object is constructed: x = 30, y = 40
A base object is constructed: i = 1, j = 2
A derived object is constructed: x = 3, y = 4
A derived2 object is constructed: w = 5, z = 6
-----
der2: 1-2-3-4-5-6
-----
Derived2ptr: 1-2-3-4-5-6
-----
Derivedptr: 100-2-200-4-5-6
-----
Baseptr: 400-2-200-4-5-6
-----
b1: 400-2
-----
d1: 400-2-200-4
-----
Derived2ptr: 11-12-13-14-15-16
-----
Derived2ptr: 400-2
-----
Derived2ptr: 400-2-200-4
-----
A derived2 object is destructed: w = 15, z = 16
A derived object is destructed: x = 13, y = 14
A base object is destructed: i = 11, j = 12
A derived object is destructed: x = 200, y = 4
A base object is destructed: i = 400, j = 2
A base object is destructed: i = 400, j = 2
Process returned 0 (0x0)   execution time : 0.203 s
```



مثال ۳: چندریختی (فایل main.cpp)

```

1  #include <iostream>
2  #include "Derived2.h"
3  using namespace std;
4
5  int main()
6  {
7      Base b1(100,200), *Baseptr;
8      Derived d1(10,20,30,40), *Derivedptr;
9      Derived2 der2(1,2,3,4,5,6), *Derived2ptr;
10     cout<<"-----\n";
11     cout << " der2: "; der2.show();
12     cout<<"-----\n";
13     Derived2ptr = &der2;
14     cout << " Derived2ptr: "; Derived2ptr -> show();
15     cout<<"-----\n";
16     Derivedptr = &der2;
17     Derivedptr -> Seti(100);
18     Derivedptr -> Setx(200);
19     //Derivedptr -> Setw(300);    Error
20     cout << " Derivedptr: "; Derivedptr -> show();
21     cout<<"-----\n";
22     Baseptr = &der2;
23     Baseptr -> Seti(400);
24     //Baseptr -> Setx(500);    Error
25     //Baseptr -> Setw(600);    Error
26     cout << " Baseptr: "; Baseptr -> show();
27     cout<<"-----\n";
28     b1 = der2;
29     cout << " b1: "; b1.show();
30     //der2 = b1;    Error
31     cout<<"-----\n";
32     d1 = der2;
33     cout << " d1: "; d1.show();
34     //der2 = d1;    Error
35     cout<<"-----\n";
36     Derived2ptr = &der2;
37     Derived2ptr -> Seti(11);
38     Derived2ptr -> Setj(12);
39     Derived2ptr -> Setx(13);
40     Derived2ptr -> Sety(14);
41     Derived2ptr -> Setw(15);
42
43     return 0;
44
45
46
47
48
49
50
51
52

```

```

"C:\Users\Maryam\courses\Advanced programing\Inheritan...
A base object is constructed: i = 100, j = 200
A base object is constructed: i = 10, j = 20
A derived object is constructed: x = 30, y = 40
A base object is constructed: i = 1, j = 2
A derived object is constructed: x = 3, y = 4
A derived2 object is constructed: w = 5, z = 6
-----
der2: 1-2-3-4-5-6
-----
Derived2ptr: 1-2-3-4-5-6
-----
Derivedptr: 100-2-200-4-5-6
-----
Baseptr: 400-2-200-4-5-6
-----
b1: 400-2
-----
d1: 400-2-200-4
-----
Derived2ptr: 11-12-13-14-15-16
-----
Derived2ptr: 400-2
-----
Derived2ptr: 400-2-200-4
-----
A derived2 object is destructed: w = 15, z = 16
A derived object is destructed: x = 13, y = 14
A base object is destructed: i = 11, j = 12
A derived object is destructed: x = 200, y = 4
A base object is destructed: i = 400, j = 2
A base object is destructed: i = 400, j = 2
Process returned 0 (0x0)   execution time : 0.203 s

```

- برنامه نوشته شده در این اسلاید، مشابه برنامه اسلاید ۱۶ است، تنها تفاوت ایجاد شده این است که تابع `show` در کلاسهای `Derived` و `Derived2` به صورت `virtual` اعلان شده است. خروجی ایجاد شده در این اسلاید را با خروجی ایجاد شده در اسلاید ۱۶ مقایسه کنید. به تفاوت نتیجه فراخوانی تابع `show` نسبت به زمانی که به صورت مجازی تعریف نشده است، دقت کنید.



نابودکننده مجازی

Virtual Destructor

- نکته دیگری که در ارتباط با توابع مجازی و چندریختی وجود دارد، تخصیص حافظه پویا برای شی از کلاس مشتق شده با استفاده از اشاره‌گری از کلاس پایه است. به عنوان مثال به کد زیر دقت کنید:

```
Base *Baseptr;
```

```
Baseptr = new Derived;
```

```
delete Baseptr;
```

- هنگام تخصیص حافظه پویا و آزاد سازی آن سازنده و نابودکننده کدام کلاس فراخوانی می‌شود؟
- در هنگام تخصیص حافظه پویا چون نوع داده کنار دستور new کلاس مشتق شده (Derived) است، سازنده کلاس مشتق شده فراخوانی می‌شود، اما هنگام آزادسازی حافظه پویا چون نوع داده اشاره‌گر از نوع کلاس پایه است نابودکننده کلاس پایه فراخوانی خواهد شد که می‌تواند عواقب نامشخصی را به دنبال داشته باشد.
- راه‌حل ساده برای جلوگیری از مشکلات ناشی از این موضوع، اعلان نابودکننده کلاس پایه به صورت مجازی است که به تبع آن نابودکننده‌های کلاس‌های مشتق شده نیز مجازی خواهند شد (حتی اگر صریحاً ذکر نشود). این کار باعث می‌شود تا هنگام آزادسازی فضای تخصیص یافته به صورت پویا مانند سایر توابع مجازی، تابع نابودکننده شی نسبت داده شده به اشاره‌گر کلاس پایه فراخوانی شود و نه نابودکننده کلاس پایه.

```
virtual ~Base();
```

- توجه داشته باشید سازنده به صورت مجازی تعریف نمی‌شود و قرار دادن کلمه کلیدی virtual کنار تابع سازنده خطا به دنبال خواهد داشت.
- برای جلوگیری از مشکلات ناشی از تخصیص حافظه پویا و آزاد سازی آن در سلسله مراتب ارث‌بری، در صورتی که حتی نیاز به نابودکننده‌ای هم برای کلاس پایه نداشته باشیم. بهتر است که تابع نابودکننده مجازی پیش‌فرض برای آن در نظر گرفت.

```
virtual ~Base() = default
```



مثال ۴: نابودکننده مجازی (فایل main.cpp)

```
1  #include <iostream>
2  #include "Derived2.h"
3  using namespace std;
4
5  int main()
6  {
7      Base *Baseptr;
8      Baseptr = new Base(1,2);
9      Baseptr ->show();
10     delete Baseptr;
11     cout<<"-----\n";
12     Baseptr = new Derived(10,20,30,40);
13     Baseptr ->show();
14     delete Baseptr;
15     cout<<"-----\n";
16     Baseptr = new Derived2(100,200,300,400,500,600);
17     Baseptr ->show();
18     delete Baseptr;
19
20     return 0;
21 }
```

```
"C:\Users\Maryam\courses\Advanced programing\Inheritan...
A base object is constructed: i = 1, j = 2
1-2
A base object is destructed: i = 1, j = 2
-----
A base object is constructed: i = 10, j = 20
A derived object is constructed: x = 30, y = 40
10-20-30-40
A base object is destructed: i = 10, j = 20
-----
A base object is constructed: i = 100, j = 200
A derived object is constructed: x = 300, y = 400
A derived2 object is constructed: w = 500, z = 600
100-200-300-400-500-600
A base object is destructed: i = 100, j = 200

Process returned 0 (0x0)   execution time : 0.156 s
Press any key to continue.
```

- اگر توابع نابودکننده مجازی تعریف نشده باشند (کلاسهای Base، Derived و Derived2 همان پیاده‌سازی اسلایدهای ۲۱ تا ۲۳ را داشته باشند)، خروجی برنامه به شکل نشان داده است. به راحتی می‌توان دید که سازنده کلاس شیء ایجاد شده فراخوانی می‌شود، در صورتی که در هر سه حالت نابودکننده کلاس پایه فراخوانی شده است.
- به خروجی تولید شده توسط تابع show نیز دقت کنید، با اینکه در سطرهای ۹، ۱۳ و ۱۷ شکل فراخوانی یکسان است، توابع متفاوتی اجرا می‌شود و خروجیهای متفاوتی ایجاد می‌شود. این همان رفتار چندشکلی در پاسخ به شکل یکسان فراخوانی است که اساس polymorphism است.



مثال ۴: نابودکننده مجازی (فایل Base.h)

```
1  #ifndef BASE_H
2  #define BASE_H
3
4  class Base
5  {
6      public:
7          Base(int = 0, int = 0);
8          void Seti(int a) {i = a;}
9          int Geti() const {return i;}
10         void Setj(int a) {j = a;}
11         int Getj() const {return j;}
12         virtual void show() const;
13         virtual ~Base();
14     protected:
15         int j;
16     private:
17         int i;
18 };
19 #endif // BASE_H
```

اعلان تابع نابودکننده مجازی ←

- فایل Base.cpp مشابه اسلاید ۵ است، زیرا در پیاده‌سازی تابع مجازی تغییر به وجود نمی‌آید.



مثال ۴: نابودکننده مجازی (فایل Derived.h)

```
1  #ifndef DERIVED_H
2  #define DERIVED_H
3
4  #include "Base.h"
5
6  class Derived:public Base
7  {
8      public:
9          Derived(int = 1,int = 1,int = 1,int = 1);
10         void Setx(int a){x = a;}
11         int Getx() const {return x;}
12         void Sety(int a){y = a;}
13         int Gety()const {return y;}
14         virtual void show() const override;
15         virtual ~Derived();
16
17     protected:
18         int y;
19
20     private:
21         int x;
22 };
23 #endif // DERIVED_H
```

اعلان نابودکننده مجازی ←

- فایل Derived.cpp مشابه اسلاید ۷ است و همانطور که در اسلاید قبلی گفته شد نیازی به تغییر در پیاده‌سازی وجود ندارد.

- ذکر کلمه کلیدی virtual برای خوانایی بهتر برنامه و حذف آن تاثیری در عملکرد برنامه ندارد.



مثال ۴: نابودکننده مجازی (فایل Derived2.h)

```
1  #ifndef DERIVED2_H
2  #define DERIVED2_H
3
4  #include "Derived.h"
5
6  class Derived2:public Derived
7  {
8      public:
9          Derived2(int = 2,int = 2, int = 2,int = 2,int = 2,int = 2);
10         void Setw(int a){w = a;}
11         int Getw() const {return w;}
12         void Setz(int a){z = a;}
13         int Getz() const {return z;}
14         virtual void show() const override;
15         virtual ~Derived2(); ← اعلان نابودکننده مجازی
16
17     protected:
18         int z;
19
20     private:
21         int w;
22 };
23
24 #endif // DERIVED2_H
```

- فایل Derived2.cpp مشابه اسلاید ۱۵ است و همانطور که در اسلایدهای قبلی گفته شد نیازی به تغییر در پیاده‌سازی وجود ندارد.
- ذکر کلمه کلیدی virtual نیز برای خوانایی بهتر برنامه است و حذف آن تاثیری در عملکرد برنامه ندارد.



مثال ۴: نابودکننده مجازی (فایل main.cpp)

```
1  #include <iostream>
2  #include "Derived2.h"
3  using namespace std;
4
5  int main()
6  {
7      Base *Baseptr;
8      Baseptr = new Base(1,2);
9      Baseptr ->show();
10     delete Baseptr;
11     cout<<"-----\n";
12     Baseptr = new Derived(10,20,30,40);
13     Baseptr ->show();
14     delete Baseptr;
15     cout<<"-----\n";
16     Baseptr = new Derived2(100,200,300,400,500,600);
17     Baseptr ->show();
18     delete Baseptr;
19
20     return 0;
21 }
```

```
"C:\Users\Maryam\courses\Advanced programming\Inheritan...
A base object is constructed: i = 1, j = 2
1-2
A base object is destructed: i = 1, j = 2
-----
A base object is constructed: i = 10, j = 20
A derived object is constructed: x = 30, y = 40
10-20-30-40
A derived object is destructed: x = 30, y = 40
A base object is destructed: i = 10, j = 20
-----
A base object is constructed: i = 100, j = 200
A derived object is constructed: x = 300, y = 400
A derived2 object is constructed: w = 500, z = 600
100-200-300-400-500-600
A derived2 object is destructed: w = 500, z = 600
A derived object is destructed: x = 300, y = 400
A base object is destructed: i = 100, j = 200

Process returned 0 (0x0)   execution time : 0.203 s
Press any key to continue.
```

- اگر توابع نابودکننده مجازی تعریف شده باشند (کلاسهای Base، Derived و Derived2 پیاده‌سازی اسلایدهای ۲۸ تا ۳۰ را داشته باشند)، خروجی برنامه به شکل نشان داده است. **خروجی این برنامه را با خروجی اسلاید ۲۷ که در آن توابع نابودکننده مجازی اعلان نشده است مقایسه کنید.**
- به راحتی می‌توان دید که در هر سه حالت سازنده و نابودکننده کلاس شیء ایجاد شده فراخوانی می‌شود. باز هم به رفتار چند شکلی در مواجهه با دستور یکسان سطرهای ۹، ۱۳ و ۱۷ و همچنین سطرهای ۱۰، ۱۴ و ۱۸ توجه کنید.



کلاسهای انتزاعی و توابع مجازی محض

Abstract Classes and Pure Virtual Functions

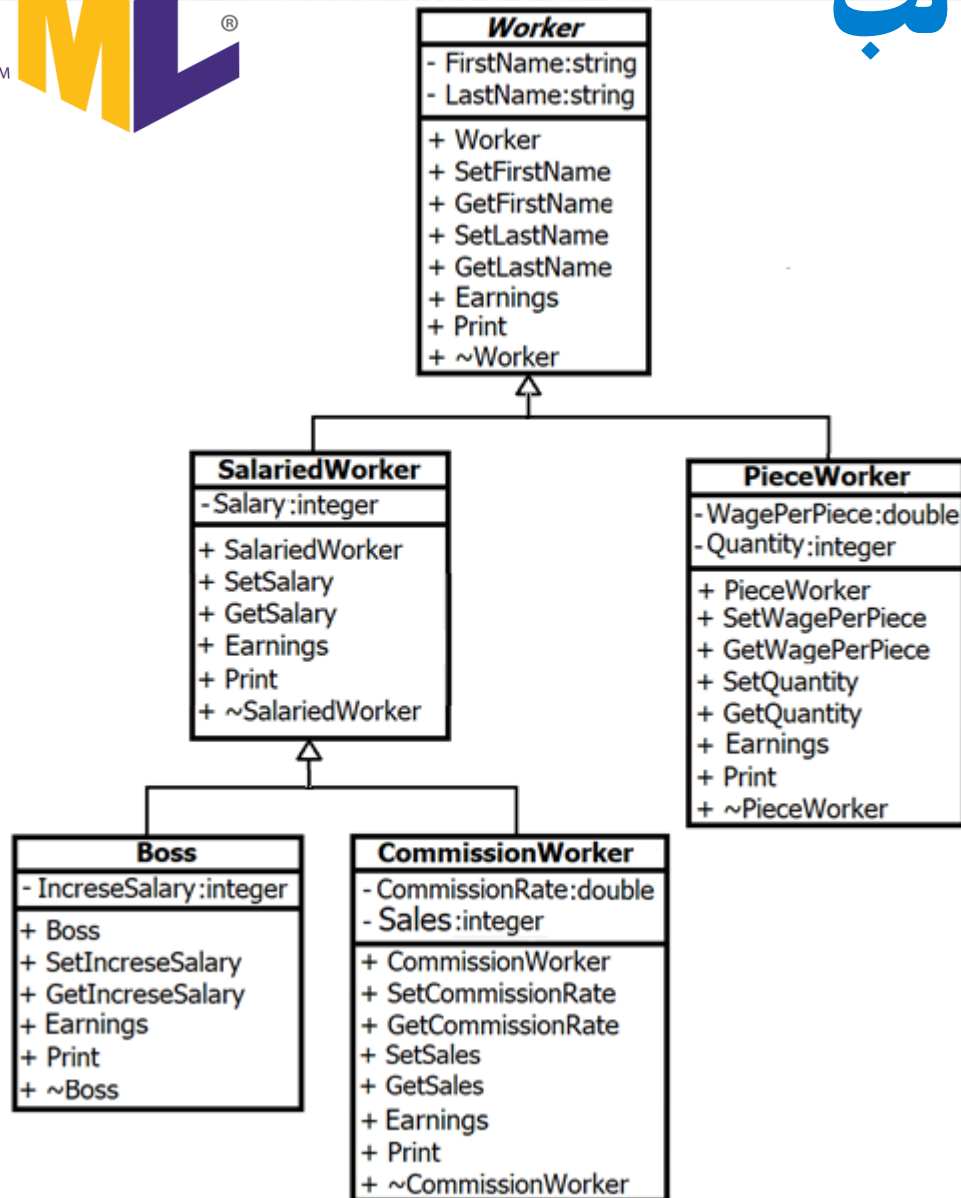
- در مثالهای بررسی شده تا کنون همه کلاسهای استفاده شده اصطلاحاً **کلاسهای عینی (concrete classes)** بودند. کلاسهای عینی کلاسهای هستند که تمامی توابع آنها دارای پیاده‌سازی هستند و در نتیجه می‌توان از آنها شیء ایجاد کرد.
- **کلاسهای انتزاعی (abstract classes)** کلاسهایی هستند که در سلسله مراتب ارث‌بری ایجاد می‌شوند و یک یا چند تابع آنها به صورت مجازی محض تعریف می‌شوند. معمولاً تابعی به صورت محض تعریف می‌شود که در کلاس پایه پیاده‌سازی برای آن متصور نیست (مثل تابع draw در کلاس shape).
- **توابع مجازی محض (pure virtual functions)** توابعی هستند که برای آنها در کلاس انتزاعی پیاده‌سازی انجام نمی‌شود و معمولاً این توابع مجازی محض در کلاسهای مشتق شده بازنویسی می‌شوند. معرفی یک تابع به عنوان تابع محض به شکل زیر انجام می‌شود:

`virtual FunctionPrototype = 0;`

- چون توابع محض پیاده‌سازی ندارند و به عبارت دیگر کلاسهای انتزاعی که دارای این توابع هستند کامل نیستند، **نمی‌توان از این کلاسها شیء ایجاد کرد؛** ولی می‌توان از این کلاسها اشاره‌گر یا ارجاع ایجاد کرد.
- کلاسهای انتزاعی معمولاً به عنوان کلاسهای پایه در سلسله مراتب ارث‌بری استفاده می‌شوند تا با بازنویسی توابع مجازی محض آنها در کلاسهای مشتق شده از مزایای ارث‌بری و چندریختی بهره‌برد.
- کلاسهای انتزاعی می‌توانند دارای اعضای داده‌ای و توابع مجازی و عادی عینی (دارای پیاده‌سازی) هم باشند که در سلسله مراتب ارث‌بری از قوانین گفته شده تا کنون پیروی می‌کنند.
- اگر در کلاس مشتق شده از کلاس پایه انتزاعی، تابع مجازی بازنویسی (پیاده‌سازی) شود، کلاس مشتق شده یک **کلاس عینی** خواهد بود و می‌توان از آن کلاس شیء ایجاد کرد.
- اگر در کلاس مشتق شده از کلاس پایه انتزاعی، تابع مجازی بازنویسی (پیاده‌سازی) نشود، کلاس مشتق شده نیز یک **کلاس انتزاعی** خواهد بود و نمی‌توان از آن کلاس شیء ایجاد کرد. ولی قاعدتاً در نهایت تابع مجازی محض باید در یکی از کلاسهای مشتق شده پیاده‌سازی شود وگرنه ایجاد کلاس پایه انتزاعی بی‌معنی خواهد بود.



مرور مطالب



می‌خواهیم سامانه پرداخت حقوق شرکتی را پیاده‌سازی کنیم. کارکنان این شرکت چهار دسته هستند. دسته اول کارکنان با حقوق ثابت هستند که ماهیانه حقوق ثابتی دارند. دسته دوم مدیران هستند که علاوه بر حقوق ثابت حق مدیریتی هم دریافت می‌کنند. دسته سوم قطعه سازانی هستند که بر اساس تعداد قطعه تولیدی و اجرتی که بابت هر قطعه تولید شده دریافت می‌کنند، حقوق آنها محاسبه می‌شود و در نهایت فروشندگان هستند که علاوه بر حقوق پایه ثابت، درصدی از کل فروش خود را نیز دریافت می‌کنند. شکل روبه‌رو نمودار کلاس مرتبط با این سامانه است.



مثال ۵: سامانه محاسبه حقوق (فایل Worker.h)

```
1  #ifndef WORKER_H
2  #define WORKER_H
3
4  #include<string>
5  #include<iostream>
6
7  using namespace std;
8
9  class Worker
10 {
11     public:
12         Worker(string, string);
13         virtual ~Worker()= default; ← اعلان تابع نابودکننده پیش فرض مجازی
14
15         string GetFirstName() const { return FirstName; }
16         void SetFirstName(string val) { FirstName = val; }
17         string GetLastName() const { return LastName; }
18         void SetLastName(string val) { LastName = val; }
19         virtual unsigned long Earnings() const = 0; ← اعلان تابع مجازی محض
20         virtual void Print() const; ← اعلان تابع مجازی
21
22     protected:
23
24     private:
25         string FirstName;
26         string LastName;
27 };
28
29 #endif // WORKER_H
```

- برابر صقر قرار دادن تابع مجازی Earnings باعث می شود که تابع Earnings مجازی محض در نظر گرفته شود و کلاس Worker هم انتزاعی به شمار آید. در این برنامه نمی توان از نوع کلاس Worker شئی ایجاد کرد.



مثال ۵: سامانه محاسبه حقوق (فایل Worker.cpp)

```
1  #include "Worker.h"
2
3  Worker::Worker(string FName, string LName) : FirstName(FName), LastName(LName)
4  {
5
6  }
7
8  void Worker::Print() const
9  {
10     cout << FirstName << ' ' << LastName;
11 }
```

- پیاده‌سازی توابع کلاس Worker که در بدنه کلاس پیاده‌سازی نشده‌اند.
- توجه کنید که با برابر قرار دادن تابع نابودکننده با **default**، نیازی به پیاده‌سازی این تابع وجود ندارد.
- توجه کنید که با صفر قرار دادن تابع **Earnings**، نیازی به پیاده‌سازی این تابع نیز وجود ندارد.



مثال ۵: سامانه محاسبه حقوق (فایل SalariedWorker.h)

```
1  #ifndef SALARIEDWORKER_H
2  #define SALARIEDWORKER_H
3
4  #include <Worker.h>
5
6  class SalariedWorker : public Worker ← ارث‌بری عمومی از کلاس Worker
7  {
8      public:
9          SalariedWorker(string, string, unsigned long = 0);
10         virtual ~SalariedWorker() = default; ← تابع نابودکننده پیش‌فرض مجازی
11
12         void SetSalary(unsigned long val) { Salary = val; }
13         unsigned long GetSalary() const { return Salary; }
14         virtual unsigned long Earnings() const override; ← بازنویسی تابع مجازی محض
15         virtual void Print() const override; ← بازنویسی تابع مجازی
16
17     protected:
18
19     private:
20         unsigned long Salary;
21 };
22
23 #endif // SALARIEDWORKER_H
```

- کلاس SalariedWorker از کلاس Worker مشتق شده است. چون تابع مجازی محض Earnings در این کلاس بازنویسی شده است، کلاس SalariedWorker کلاسی عینی (نه انتزاعی) است و می‌توان از آن شیء ایجاد نمود.



مثال ۵: سامانه محاسبه حقوق (فایل SalariedWorker.cpp)

```
1  #include "SalariedWorker.h"
2
3  SalariedWorker::SalariedWorker(string FName, string LName, unsigned long s):
4      Worker(FName, LName), Salary(s)
5  {
6  }
7
8  unsigned long SalariedWorker::Earnings() const
9  {
10     return GetSalary();
11 }
12
13 void SalariedWorker::Print() const
14 {
15     cout << "Salaried Worker: ";
16     Worker::Print();
17 }
```

فراخوانی سازنده کلاس پایه

بازنویسی تابع مجازی محض

بازنویسی تابع مجازی

فراخوانی تابع کلاس پایه

- همه توابع کلاس SalariedWorker غیر از نابودکننده پیاده‌سازی شده‌اند.
- در صورت عدم پیاده‌سازی تابع Earnings، این کلاس هم کلاسی انتزاعی می‌شد.
- در صورت عدم پیاده‌سازی تابع Print، این کلاس پیاده‌سازی تابع Print کلاس Worker را به ارث می‌برد.



مثال ۵: سامانه محاسبه حقوق (فایل PieceWorker.h)

```
1  #ifndef PIECEWORKER_H
2  #define PIECEWORKER_H
3
4  #include <Worker.h>
5
6  class PieceWorker : public Worker ← ارث‌بری عمومی از کلاس Worker
7  {
8      public:
9          PieceWorker(string, string, unsigned int = 0, unsigned int = 0);
10         virtual ~PieceWorker() = default; ← تابع نابودکننده پیش‌فرض مجازی
11
12         unsigned int GetWagePerPiece() const { return WagePerPiece; }
13         void SetWagePerPiece(unsigned int val) { WagePerPiece = val; }
14         unsigned int GetQuantity() const { return Quantity; }
15         void SetQuantity(unsigned int val) { Quantity = val; }
16         virtual unsigned long Earnings() const override; ← بازنویسی تابع مجازی محض
17         virtual void Print() const override; ← بازنویسی تابع مجازی
18
19     protected:
20
21     private:
22         unsigned int WagePerPiece;
23         unsigned int Quantity;
24 };
25
26 #endif // PIECEWORKER_H
```

- کلاس PieceWorker از کلاس Worker مشتق شده است. چون تابع مجازی محض Earnings در این کلاس بازنویسی شده است، کلاس PieceWorker کلاسی عینی (نه انتزاعی) است و می‌توان از آن شیء ایجاد نمود.



مثال ۵: سامانه محاسبه حقوق (فایل PieceWorker.cpp)

```
1  #include "PieceWorker.h"
2
3  PieceWorker::PieceWorker(string FName, string LName, unsigned int w, unsigned int q):
4      Worker(FName, LName), WagePerPiece(w), Quantity(q) ← فراخوانی سازنده کلاس پایه
5  {
6  }
7
8  unsigned long PieceWorker::Earnings() const ← بازنویسی تابع مجازی محض
9  {
10     return Quantity * WagePerPiece;
11 }
12
13 void PieceWorker::Print() const ← بازنویسی تابع مجازی
14 {
15     cout << "Piece Worker: ";
16     Worker::Print(); ← فراخوانی تابع کلاس پایه
17 }
```

- همه توابع کلاس PieceWorker غیر از نابودکننده پیاده‌سازی شده‌اند.
- در صورت عدم پیاده‌سازی تابع Earnings، این کلاس هم کلاسی انتزاعی می‌شد.
- در صورت عدم پیاده‌سازی تابع Print، این کلاس پیاده‌سازی تابع Print کلاس Worker را به ارث می‌برد.



مثال ۵: سامانه محاسبه حقوق (فایل Boss.h)

```
1  #ifndef BOSS_H
2  #define BOSS_H
3
4  #include <SalariedWorker.h>
5
6  class Boss : public SalariedWorker ← ارث‌بری عمومی از کلاس SalariedWorker
7  {
8      public:
9          Boss(string, string, unsigned long = 0, unsigned int = 0);
10         virtual ~Boss() = default; ← تابع نابودکننده پیش‌فرض مجازی
11
12         unsigned int GetIncreaseSalary() const { return IncreaseSalary; }
13         void SetIncreaseSalary(unsigned int val) { IncreaseSalary = val; }
14         virtual unsigned long Earnings() const override; ← بازنویسی تابع مجازی
15         virtual void Print() const override; ← بازنویسی تابع مجازی
16
17     protected:
18
19     private:
20         unsigned int IncreaseSalary;
21 };
22
23 #endif // BOSS_H
```

- کلاس Boss از کلاس SalariedWorker مشتق شده است. کلاس Boss کلاسی عینی (نه انتزاعی) است و می‌توان از آن شیء ایجاد نمود.



مثال ۵: سامانه محاسبه حقوق (فایل Boss.cpp)

```
1  #include "Boss.h"
2
3  Boss::Boss(string FName, string LName, unsigned long s, unsigned int i):
4      SalariedWorker(FName, LName, s), IncreaseSalary(i)
5  {
6  }
7
8  unsigned long Boss::Earnings() const
9  {
10     return SalariedWorker::GetSalary() + IncreaseSalary;
11 }
12
13 void Boss::Print () const
14 {
15     cout << "Boss: ";
16     Worker::Print();
17 }
18
```

فراخوانی سازنده کلاس پایه ←

← بازنویسی تابع مجازی

← بازنویسی تابع مجازی

← فراخوانی تابع کلاس پایه غیرمستقیم

- همه توابع کلاس Boss غیر از نابودکننده پیاده‌سازی شده‌اند.
- در صورت عدم پیاده‌سازی تابع Earnings، این کلاس پیاده‌سازی تابع Earnings کلاس SalariedWorker را به ارث می‌برد.
- در صورت عدم پیاده‌سازی تابع Print، این کلاس پیاده‌سازی تابع Print کلاس SalariedWorker را به ارث می‌برد.



مثال ۵: سامانه محاسبه حقوق (فایل CommissionWorker.h)

```
1  #ifndef COMMISSIONWORKER_H
2  #define COMMISSIONWORKER_H
3
4  #include <SalariedWorker.h>
5
6  class CommissionWorker : public SalariedWorker ← ارث‌بری عمومی از کلاس SalariedWorker
7  {
8      public:
9          CommissionWorker(string, string, unsigned long = 0, double = 0.05, unsigned long = 0);
10         virtual ~CommissionWorker() = default; ← تابع نابودکننده پیش‌فرض مجازی
11
12         double GetCommissionRate() const { return CommissionRate; }
13         void SetCommissionRate(double val);
14         unsigned long GetSales() const { return Sales; }
15         void SetSales(unsigned long val) { Sales = val; }
16         virtual unsigned long Earnings() const override; ← بازنویسی تابع مجازی
17         virtual void Print() const override; ← بازنویسی تابع مجازی
18
19     protected:
20
21     private:
22         double CommissionRate;
23         unsigned long Sales;
24 };
25
26 #endif // COMMISSIONWORKER_H
```

- کلاس CommissionWorker از کلاس SalariedWorker مشتق شده است. کلاس CommissionWorker کلاسی عینی است و می‌توان از آن شیء ایجاد نمود.



مثال ۵: سامانه محاسبه حقوق (فایل CommissionWorker.cpp)

```
1  #include "CommissionWorker.h"
2
3  CommissionWorker::CommissionWorker(string FName, string LName, unsigned long s, double r, unsigned long sa):
4      SalariedWorker(FName, LName, s), Sales(sa)
5  {
6      SetCommissionRate(r);
7  }
8
9  void CommissionWorker::SetCommissionRate(double val)
10 {
11     if (0 <= val && val <= 1)
12         CommissionRate = val;
13     else
14         CommissionRate = 0.05;
15 }
16
17 unsigned long CommissionWorker::Earnings() const
18 {
19     return SalariedWorker::GetSalary() + Sales * CommissionRate;
20 }
21
22 void CommissionWorker::Print() const
23 {
24     cout << "Commission Worker: ";
25     Worker::Print();
26 }
```

فراخوانی سازنده کلاس پایه

بازنویسی تابع مجازی

بازنویسی تابع مجازی

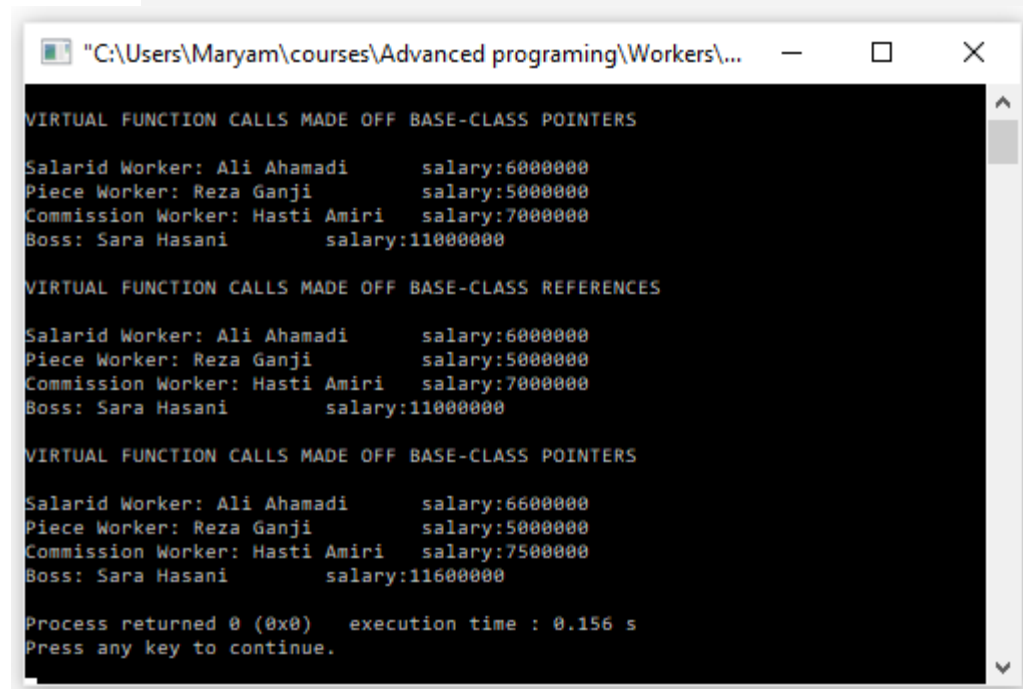
فراخوانی تابع کلاس پایه غیرمستقیم

- همه توابع کلاس CommissionWorker غیر از نابودکننده پیاده‌سازی شده‌اند.
- در صورت عدم پیاده‌سازی تابع Earnings، این کلاس پیاده‌سازی تابع Earnings کلاس SalariedWorker را به ارث می‌برد.
- در صورت عدم پیاده‌سازی تابع Print، این کلاس پیاده‌سازی تابع Print کلاس SalariedWorker را به ارث می‌برد.



مثال ۵: سامانه محاسبه حقوق (فایل main.cpp)

```
1  #include <iostream>
2  #include <vector>
3  #include "SalariedWorker.h"
4  #include "PieceWorker.h"
5  #include "CommissionWorker.h"
6  #include "Boss.h"
7
8  using namespace std;
9
10 void PrintEarningsViaPointer(Worker *);
11 void PrintEarningsViaReference(Worker &);
12 int main()
13 {
14     //Worker w("Maryam", "Khorashadizade");      Error
15     SalariedWorker sw("Ali", "Ahamadi", 6000000);   PieceWorker pw("Reza", "Ganji", 100, 50000);
16     CommissionWorker cw("Hasti", "Amiri", 5000000, 0.1, 2000000);   Boss b("Sara", "Hasani", 6000000, 5000000);
17
18     vector <Worker *> workers = {&sw, &pw, &cw, &b};
19
20     cout << "WORKERS PROCESSED POLYMORPHICALLY VIA DYNAMIC BINDING\n";
21     cout << "\nVIRTUAL FUNCTION CALLS MADE OFF BASE-CLASS POINTERS\n\n";
22     for(Worker *Wptr:workers)
23         PrintEarningsViaPointer(Wptr);
24     cout << "\nVIRTUAL FUNCTION CALLS MADE OFF BASE-CLASS REFERENCES\n\n";
25     for(Worker *Wptr:workers)
26         PrintEarningsViaReference(*Wptr);
27
28     SalariedWorker *SalariedWorkerPtr;
29     for(Worker *Wptr:workers)
30     {
31         SalariedWorkerPtr = dynamic_cast<SalariedWorker *>(Wptr);
32         if(SalariedWorkerPtr!=0)
33             SalariedWorkerPtr -> SetSalary(SalariedWorkerPtr -> GetSalary() *1.1);
34     }
35     cout << "\nVIRTUAL FUNCTION CALLS MADE OFF BASE-CLASS POINTERS\n\n";
36     for(Worker *Wptr:workers)
37         PrintEarningsViaPointer(Wptr);
38
39     return 0;
40 }
41 void PrintEarningsViaPointer(Worker *w)
42 {
43     w -> Print();
44     cout<<"\t salary:"<<w -> Earnings() << endl;
45 }
46
47 void PrintEarningsViaReference(Worker &w)
48 {
49     w.Print();
50     cout<<"\t salary:"<<w.Earnings() << endl;
51 }
```



```
"C:\Users\Maryam\courses\Advanced programming\Workers\...
VIRTUAL FUNCTION CALLS MADE OFF BASE-CLASS POINTERS
Salarid Worker: Ali Ahamadi      salary:6000000
Piece Worker: Reza Ganji        salary:5000000
Commission Worker: Hasti Amiri   salary:7000000
Boss: Sara Hasani               salary:11000000

VIRTUAL FUNCTION CALLS MADE OFF BASE-CLASS REFERENCES
Salarid Worker: Ali Ahamadi      salary:6000000
Piece Worker: Reza Ganji        salary:5000000
Commission Worker: Hasti Amiri   salary:7000000
Boss: Sara Hasani               salary:11000000

VIRTUAL FUNCTION CALLS MADE OFF BASE-CLASS POINTERS
Salarid Worker: Ali Ahamadi      salary:6600000
Piece Worker: Reza Ganji        salary:5500000
Commission Worker: Hasti Amiri   salary:7700000
Boss: Sara Hasani               salary:11600000

Process returned 0 (0x0)   execution time : 0.156 s
Press any key to continue.
```

سامانه محاسبه حقوق

- در مثال سامانه محاسبه حقوق اشیایی از نوع کلاسهای عینی ایجاد شده است (سطرهای ۱۵ و ۱۶). توجه داشته باشید که ایجاد شیء از نوع کلاس انتزاعی **Worker** خطا به دنبال خواهد داشت.
- در ادامه برداری (vector) از نوع اشاره گر به کلاس پایه **Worker** ایجاد شده است و با آدرس اشیای کلاسهای مشتق شده مقداردهی می شود (سطر ۱۸). سپس برای تمامی عناصر بردار تابع **PrintEarningsViaPointer** فراخوانی شده است (سطر ۲۳).
- در هر فراخوانی تابع **PrintEarningsViaPointer** توابع مجازی **Print** و **Earnings** از طریق اشاره گر کلاس پایه فراخوانی می شود. در این تابع هیچ تغییری از نوع کلاسهای مشتق شده وجود ندارد و تنها از اشاره گر به کلاس پایه **Worker** استفاده شده است. در زمان کامپایل مشخص نیست که چه کدی در فراخوانی توابع مجازی **Print** و **Earnings** اجرا خواهد شد؛ اما در زمان اجرا با توجه به ساختمان داده های ایجاد شده توسط کامپایلر برای پشتیبانی چند ریختی، توابع مناسب به فراخوانیهای توابع مجازی متصل شده (dynamic binding) و اجرا می شوند. همانطوری که در خروجی مشاهده می کنید، علیرغم شکل یکسان فراخوانی توابع مجازی (سطرهای ۴۳ و ۴۴)، توابع مختلفی اجرا و نتایج به درستی چاپ شده است.
- در سطرهای ۲۵ و ۲۶ مشابه عملیات قبلی ولی این بار از طریق متغیرهای ارجاع کلاس پایه انجام شده است. در تابع **PrintEarningsViaReference** توابع مجازی **Print** و **Earnings** از طریق متغیر ارجاع کلاس پایه فراخوانی شده است و مشابه حالت قبلی با رفتار چندریختی علیرغم شکل یکسان فراخوانی توابع مجازی (سطرهای ۴۹ و ۵۰)، توابع مختلفی اجرا و نتایج به درستی چاپ شده است.
- دقت کنید برای افزودن کارکنان جدید به این سامانه کافی است که کارکنان جدید به بردار **Workers** اضافه شوند و نیازی به تغییر تابع **PrintEarningsViaPointer** و یا **PrintEarningsViaReference** و نحوه فراخوانی آنها نیست. توجه کنید حتی می توان دسته جدیدی از کارکنان با شکل دیگری از محاسبه حقوق (یک کلاس جدید) ایجاد کرد، در صورتی که کلاس جدید همچنان از کلاس پایه **Worker** ارث بری کند، باز هم نیازی به تغییر تابع **PrintEarningsViaPointer** و یا **PrintEarningsViaReference** و نحوه فراخوانی آنها نیست. به همین دلیل است که چندریختی با عمومیت بخشی به برنامه، توسعه برنامه ها را آسانتر می کند.
- در استفاده از چندریختی باید این نکته را در نظر داشت که ایجاد ساختمان داده های لازم و مقداردهی آنها برای اجرای صحیح چندریختی، خود زمان و حافظه صرف می کند، که این مساله شاید در مواردی باعث عدم تمایل به استفاده از چندریختی در برنامه نویسی شود.



سامانه محاسبه حقوق

- در ادامه مثال سامانه محاسبه حقوق فرض شده است که می‌خواهیم حقوق پایه تمامی کارکنان را ۱۰ درصد افزایش دهیم. دقت کنید که این افزایش حقوق شامل افرادی که حقوق ثابت ندارند، یعنی قطعه‌سازان نخواهد شد. چون ما از چندریختی در این برنامه استفاده کردیم، بردار `Workers` که اطلاعات کارکنان را نگهداری می‌کند، شامل اطلاعات همه کارکنان است در صورتی که در این مثال ما باید کارکنانی که حقوق ثابت دارند، مشخص کنیم. برای این کار از عملگر `dynamic_cast` استفاده شده است.
- در مثالهای قبلی دیدید که نسبت دادن آدرس شیء کلاس پایه (یا اشاره‌گر کلاس پایه) به اشاره‌گر کلاس مشتق شده خطا به دنبال خواهد داشت، مگر اینکه این کار صریحا با عملگرهای **تغییر نوع داده (type casting)** انجام شود. در اسلاید ۲۴ این کار با عملگر `static_cast` انجام شده بود که البته این کار مخاطراتی نیز به همراه دارد. به عنوان مثال اگر در این مثال به جای `dynamic_cast` از `static_cast` استفاده شود با توجه به آنکه با استفاده از اشاره‌گر کلاس مشتق شده، می‌توان توابعی که منحصر در کلاس مشتق شده وجود دارد و در کلاس پایه وجود ندارد (مانند `SetSalary` و `GetSalary`)، را فراخوانی کند، این عمل نتایج پیش‌بینی نشده‌ای را برای اشیای دیگر (مانند `pw`) به دنبال دارد.
- در اینجا از عملگر `dynamic_cast` استفاده شده است. ابتدا اشاره‌گری از نوع کلاس `SalariedWorker` ایجاد شده است (سطر ۲۸- اشاره‌گر `SalariedWorkerPtr`)؛ سپس به این اشاره‌گر با عملگر `dynamic_cast`، عناصر بردار `Workers` که از نوع اشاره‌گر کلاس پایه است، نسبت داده شده است (سطر ۳۱). دقت کنید همانطور که گفتیم این کار بدون تغییر نوع داده (`type casting`) امکانپذیر نیست. اما تفاوتی که `dynamic_cast` با `static_cast` دارد، این است که تنها در صورتی این انتساب انجام می‌شود که درون اشاره‌گر کلاس پایه، آدرس شیء از کلاس `SalariedWorker` قرار گرفته باشد، در غیر این صورت اشاره‌گر پوچ (`nullptr`) به `SalariedWorkerPtr` نسبت داده می‌شود. بدین شکل با مقایسه اشاره‌گر `SalariedWorkerPtr` با اشاره‌گر پوچ می‌توان تعیین کرد که آیا به شیء از نوع کلاس `SalariedWorker` اشاره می‌کند یا نه (سطر ۳۲). در صورتی که اشاره‌گر به شیء از نوع کلاس `SalariedWorker` اشاره کند، با فراخوانی توابع `GetSalary` مقدار فعلی حقوق دریافت شده و پس از افزایش ۱۰ درصدی با استفاده از تابع `SetSalary` مقدار جدید ثبت می‌شود (سطر ۳۳) و در صورتی که درون اشاره‌گر پایه آدرس شیء غیر از شیء `SalariedWorker` قرار داشته باشد، اشاره‌گر پوچ می‌شود و با توجه به شرط دیگر توابع `SetSalary` و `GetSalary` فراخوانی نمی‌شود و جلوی تغییرات ناخواسته گرفته می‌شود.
- در ادامه دوباره مقادیر حقوق کارکنان با استفاده از بردار `Workers` و تابع `PrintEarningsViaPointer` چاپ شده است. دقت کنید که تغییرات به شکل درستی روی کارکنان دارای حقوق پایه (کلاس `SalariedWorker` و کلاسهای مشتق شده از آن) و نه کارکنان دیگر اعمال شده است. علت نامگذاری `dynamic_cast` این است که در زمان اجرا تغییر نوع داده و انتساب انجام می‌شود.

